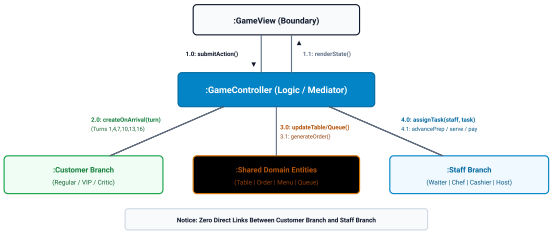
CT2003 Restaurant Rush

System Communication Diagram & Decoupled Architecture Guide

STAGE 1 - 3 ARCHITECTURE

This document details the object communication flow and decoupled state/behavior specifications for the **CT2003 Restaurant Rush** simulation. The design enforces strict tier separation: **Customer Branch** and **Staff Branch** remain 100% decoupled, communicating exclusively through the **GameController** and shared domain entities.

1. UML Communication Diagram (Numbered Message Sequence)



2. Key Architectural Principles

Decoupled Customer Generation

Customer objects are instantiated **dynamically by the Logic layer** on specific arrival turns (1, 4, 7, 10, 13, 16). They enter the queue with 100 satisfaction.

- 1 Table holds exactly **1 Customer** at a time.
- Staff do not reference Customer instances directly.

Mediated Task Execution

The **GameController** evaluates player choices and dispatches actions to Staff based on Table and Order states.

- **Waiter:** 1 task/turn (Seat, Take Order, Serve, Wait).
- **Chef:** 1 prep unit/turn.
- **Cashier:** 1 payment/turn.

> **Timing Precision Rule:** An order prepped on Turn *t* transitions to READY. It can be served on Turn *t+1* at earliest (SERVED), eaten on Turn *t+2* (READY_TO_PAY), and collected on Turn *t+3* (PAID).

3. Class Specifications: Logic & Shared Entities

Class / Module	State (Attributes)	Behavior (Methods / Responsibilities)
GameController `Logic Layer`	• `currentTurn: int` (1 to 18) • `totalRevenue: int` (Cents) • `happyHourTurns: int` • `restaurant: Restaurant`	• `runTurn(waiterChoice, chefChoice, hostChoice)` • `checkAndSpawnArrivals(turn)` • `processSatisfactionDecay()` • `evaluateGameOutcome(): GameResult`
Table `Entity`	• `tableNumber: int` • `occupiedBy: Customer` • `isOccupied: boolean`	• `assignCustomer(c: Customer): void` • `clearTable(): void` • *Rule: Maximum capacity = 1 Customer*
Order `Entity`	• `item: MenuItem` • `unitsDone: int` • `requiredUnits: int` • `status: OrderStatus`	• `advancePrep(): void` • `isReady(): boolean` • Status: `PREPARING` → `READY` → `SERVED` → `PAID`
BillCalculator `Domain Service`	• `vipDiscount: double` (10%) • `happyHourDiscount: double` (20%)	• `calculateBill(basePrice, customer, isHappyHour): int` • Applies tier discounts without mutating customer state.

4. Class Specifications: Customer & Staff Branches

Branch Class	State (Attributes)	Behavior (Methods / Polymorphism)
Customer `Abstract Subtree`	• `id: String` • `satisfaction: int` (Init: 100) • `status: CustomerStatus`	• `abstract decaySatisfaction(): void` • `abstract getDiscountRate(): double` • `chooseFood(menu: Menu): MenuItem`
RegularCustomer `Stage 1`	• `decayRate: int = 8` per turn	• `decaySatisfaction()`: -8 satisfaction/turn
VIPCustomer `Stage 2`	• `decayRate: int = 5` per turn • `discount: double = 0.10`	• `decaySatisfaction()`: -5 satisfaction/turn • `getDiscountRate()`: Returns 0.10
CriticCustomer `Stage 2`	• `decayRate: int = 12` per turn	• `decaySatisfaction()`: -12 satisfaction/turn • Influences end-game scoring rating
Waiter `Staff Branch`	• `currentTask: WaiterTask`	• `seatCustomer()`, `takeOrder()`, `serveOrder()`
Chef `Staff Branch`	• `assignedOrder: Order`	• `cookOrder(order)`: Increments prep units by 1
Cashier `Staff Branch`	• `isBusy: boolean`	• `processPayment(table)`: Auto-clears bill & frees table
Host `Stage 3 Staff`	• `seatsThisTurn: int`	• `seatExtraCustomer()`: Seats 1 extra customer/turn

Stage Submission Checklist & Tag Guidelines

stage-1: Regular customers only, basic Menu, Waiter, Chef, Cashier, 2 Tables, 18-turn loop. No Host/VIP/HappyHour.
stage-2: Introduce VIP & Critic subclasses (replace arrivals at turns 7 and 13).
stage-3: Add Host staff, ComboMeal composition, and Happy Hour boost mechanics.